

# An Autonomous Robot for Indoor Navigation Using A\* Search: A Software-Only Simulation with Dynamic Obstacle Replanning

Md. Sadman Bin Murtoza, MD Asraful Hossain Riyan, Fahima Shameem, Sanjida Jaman  
Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh  
Group 6, CSE440 Section 2

**Abstract**—This report presents the design, implementation, and evaluation of a software-only simulation of an autonomous indoor navigation robot. The system represents an indoor environment as a two-dimensional occupancy grid and uses the A\* search algorithm with a Manhattan-distance heuristic to compute the shortest safe path between a user-selected start and destination cell. A virtual robot then follows the computed path cell by cell inside an interactive Tkinter graphical interface, and the system automatically detects and safely reacts to obstacles that are added dynamically while the robot is moving, replanning a new route from the robot's current position. The project was implemented in pure Python with no external hardware, organized into four co-operating modules developed individually by the four members of Group 6 and integrated through a single connector script. We describe the occupancy-grid data model, the A\* planning core, the virtual-robot state machine, and the graphical interface, and we report empirical results, including path-planning time, explored-node counts, and an automated regression-test suite, obtained by executing the group's own source code on a range of grid sizes and obstacle layouts, including a full reachable-to-unreachable dynamic replanning scenario. The results confirm that the implemented A\* planner consistently returns the optimal (shortest) path when one exists, correctly reports an unreachable destination, and replans successfully around newly introduced obstacles in under one millisecond on a 15×20 grid.

**Index Terms**—A\* search, heuristic search, path planning, occupancy grid, autonomous navigation, dynamic replanning, Manhattan distance, Tkinter simulation.

## I. INTRODUCTION

Autonomous indoor navigation is a foundational problem in mobile robotics and artificial intelligence: given a map of free and occupied space, a robot must compute a collision-free route from its current position to a goal, and it must react safely when the environment changes while it is moving. Real deployments of this problem involve sensing, localization, and control hardware; however, the core *planning* problem, deciding which sequence of grid cells to traverse, can be studied and demonstrated rigorously without any physical robot. This project takes that approach.

Group 6 designed and implemented a fully software-based simulation in which an indoor floor is represented as a two-dimensional occupancy grid, free cells are traversable and blocked cells are not, and a virtual robot moves through the grid one cell at a time along a route produced by the A\* search algorithm [1]. The system additionally supports *dynamic obstacles*: a user may add a new obstacle to the grid while the robot is en route, and the robot is required to stop safely before entering the blocked cell and to trigger a fresh A\* search from its current position rather than from the original start cell.

### A. Motivation and Scope

The motivation for a simulation-only scope was twofold. First, it allowed the group to focus its limited semester time on the algorithmic and software-engineering aspects of navigation, path planning, state management, dynamic replanning, and user interaction, rather than on sensor calibration and motor control. Second, it produces a self-contained, easily demonstrable, and easily graded artifact: the complete system runs from a single Python entry point with no external hardware or paid services.

### B. Objectives

The specific objectives of the project were to: (1) represent an indoor environment as an occupancy grid supporting obstacle placement and removal; (2) implement A\* search with an admissible heuristic to compute the shortest safe path between any two free cells; (3) animate a virtual robot following that path with a visibly paced, cell-by-cell motion; (4) detect obstacles introduced after planning and safely halt before entering a newly blocked cell; (5) automatically replan from the robot's current position when this occurs; and (6) present the entire process through an interactive graphical interface with a live performance dashboard and an automated self-test suite.

### C. Problem Statement

Formally, given a grid  $G$  of free and occupied cells, a start cell  $s$ , and a goal cell  $g$ , the system must (i) compute a sequence of unit moves  $s = p_0, p_1, \dots, p_k = g$  such that every  $p_i$  is free and every consecutive pair is four-connected, while minimizing  $k$ ; (ii) execute that sequence visibly over time; and (iii) if some cell  $p_j$  ( $j >$  the robot's current index) is occupied by a newly added obstacle before the robot reaches it, halt at  $p_{j-1}$  and recompute a new minimal sequence from  $p_{j-1}$  to  $g$  over the updated grid, repeating step (ii)–(iii) until either the goal is reached or no such sequence exists. Sections IV and V describe, respectively, how this problem is solved and how the solution was verified against the group's own implementation.

### D. Report Organization

Section II summarizes related approaches to grid-based path planning. Section III describes the overall system architecture, module responsibilities, and the mapping from user interaction to module calls. Section IV details the methodology and implementation of each module, including the exact algorithm used for path planning. Section V reports experimental results obtained directly from the group's source code, including a scalability sweep and a comparison against uninformed search. Section VI discusses complexity, design trade-offs, limitations, and reproducibility. Section VII documents individual member contributions against the group's seven-week timeline, and Section VIII concludes with a summary and directions for future work.

## II. RELATED WORK

Grid-based path planning is commonly addressed with graph-search algorithms. Uninformed methods such as breadth-first search and Dijkstra’s algorithm guarantee shortest paths but expand nodes without regard to their proximity to the goal, which is inefficient on large open grids. A\* search improves on this by combining the real cost-so-far,  $g(n)$ , with an admissible heuristic estimate of the remaining cost,  $h(n)$ , and expanding nodes in order of  $f(n) = g(n) + h(n)$  [1, 2]. When the heuristic never overestimates the true remaining cost, A\* is guaranteed to return an optimal path while typically exploring far fewer nodes than uninformed search on the same grid.

For environments that change over time, such as a corridor that becomes blocked after a path has already been computed, incremental and replanning variants of A\* have been proposed in the robotics literature, most notably D\* Lite, which reuses information from a previous search to replan efficiently after localized map changes [3]. The present project does not implement an incremental search structure; instead, it adopts the simpler and easier-to-verify strategy of re-invoking a complete A\* search from the robot’s current cell whenever the next planned cell becomes blocked. This design choice is discussed further in Section VI.

Manhattan distance (also called taxicab distance) is a standard admissible and consistent heuristic for grids restricted to four-directional (up/down/left/right) movement, since it exactly equals the minimum number of moves required in the absence of obstacles [2, 4]. It was therefore selected for the group’s implementation, described in Section IV. An alternative, 8-connected formulation using an octile-distance heuristic (permitting diagonal movement) was explored by the group during early architecture design, since diagonal movement can shorten paths in open areas; this alternative was not carried into the final integrated system in favor of the simpler, easier-to-verify 4-connected formulation, which is discussed further in Section VI-C.

Beyond the specific search algorithms compared above, indoor mobile-robot navigation in practice is often addressed with full navigation stacks, such as the ROS Navigation Stack used with platforms like TurtleBot, which combine a global grid-based planner conceptually similar to the one implemented here with a local reactive-avoidance layer, simultaneous localization and mapping (SLAM) for building the occupancy grid from sensor data, and closed-loop motor control. These broader system-level design options, including compute and sensing choices, were surveyed by the group during early architecture planning, informed by a review of classical graph search, sampling-based planners such as RRT/RRT\*, and reactive methods such as potential fields, before the software-only, single-global-planner scope of the present system was deliberately fixed to keep the semester-long deliverable focused and fully verifiable without physical hardware.

## III. SYSTEM ARCHITECTURE

The system is organized into four cooperating modules, one per group member, connected by a single integration script. Fig. 1 shows the data flow between modules, and Table I lists the source files, and their responsibilities.

The design follows a clear separation of concerns: Member 1’s GridMap class owns only the static and semi-static map state (the 0/1 grid, start, and goal); Member 2’s AStarPlanner

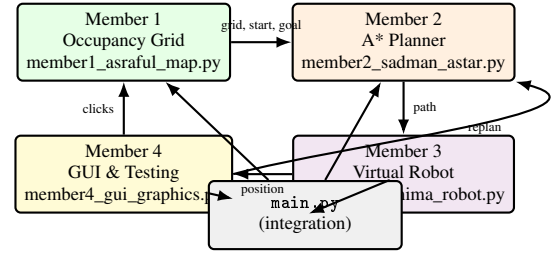


Fig. 1. Software architecture of the simulator. `main.py` instantiates the grid, planner, and robot objects and passes them to the GUI, which coordinates all inter-module calls at runtime.

TABLE I  
MODULE RESPONSIBILITIES AND OWNERSHIP

| Member                | File                               | Responsibility                                                                                                   |
|-----------------------|------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Asraful Hossain Riyan | member1_asraful_map.py             | Occupancy grid, obstacle add/remove, start/goal selection, map save/load (JSON)                                  |
| Sadman Bin Murtoza    | member2_sadman_astar.py<br>main.py | A* shortest-path search, path validation, performance metrics, replanning, system integration                    |
| Fahima Shameem        | member3_fahima_robot.py            | Virtual robot state machine: step-by-step movement, travelled/remaining cells, pause/resume, blocked-cell safety |
| Sanjida Jaman         | member4_gui_graphics.py            | Tkinter GUI, color-coded rendering, control panel, performance dashboard, automated system tests                 |

class is a stateless search engine that takes a grid and two cells and returns a result dictionary; Member 3’s VirtualRobot class owns only the robot’s dynamic state (current position, travelled/remaining cells, running/paused/finished flags); and Member 4’s RobotGUI class is the only module that imports Tkinter and is responsible for wiring user input to the other three modules and rendering their state each frame. `main.py` contains no logic beyond object construction and is the single point where all four modules are joined, exactly as required by the course’s module-ownership guidelines.

### A. User Workflow and Control Mapping

Table II maps each GUI control to the module method it ultimately invokes, illustrating how a user action flows through the layered architecture of Fig. 1. The demonstration workflow documented in the project’s README.md exercises every row of this table in sequence: choose start, choose destination, add obstacles, find a path, run the robot, add a new obstacle on the active route, and observe the automatic stop-and-replan.

## IV. METHODOLOGY AND IMPLEMENTATION

### A. Occupancy Grid Representation

The environment is modeled by the GridMap class as a rectangular list-of-lists grid of  $\text{rows} \times \text{columns}$  cells (the GUI initializes a  $15 \times 20$  grid, i.e., 300 cells, by default in `main.py`), where a cell value of 0 denotes free space and 1 denotes an obstacle. The class exposes `add_obstacle`, `remove_obstacle`, `set_start`, and `set_goal`, each of which validates that the target cell lies inside the grid before mutating state, and `set_start/set_goal` additionally reject blocked cells so that the robot can never be asked to start or end inside a wall. `add_obstacle` likewise refuses to overwrite the currently selected start or goal cell. The map can be serialized

TABLE II  
GUI CONTROLS AND THE MODULE METHODS THEY INVOKE

| Control                                    | Target module | Method invoked                   |
|--------------------------------------------|---------------|----------------------------------|
| Choose Start / Destination                 | Member 1      | set_start / set_goal             |
| Add / Remove Obstacle                      | Member 1      | add_obstacle / remove_obstacle   |
| Find A* Path                               | Member 2      | find_path                        |
| Run Robot                                  | Member 3      | load_path, start                 |
| Pause / Resume                             | Member 3      | pause / resume                   |
| Stop / Reset                               | Member 3      | stop / reset                     |
| Simulation (automatic on obstacle-in-path) | Members 2 & 3 | replan then apply_replanned_path |
| Save Map / Load Map                        | Member 1      | save / load                      |
| Run System Tests                           | Members 1–3   | see Section IV-E                 |

to and restored from a JSON file through save and load, with load performing shape and value validation (rectangular grid, cells restricted to  $\{0, 1\}$ ) before accepting an external file.

1) *Data Structure Choice* The grid is stored as a plain Python list-of-lists rather than, for example, a NumPy array or a sparse set of obstacle coordinates. This was a deliberate trade-off: a dense list-of-lists makes inside, add\_obstacle, and cell look-ups inside the A\* neighbor expansion simple  $O(1)$  index operations with no external dependency (requirements.txt lists no third-party packages for the final GUI version), at the cost of using  $O(\text{rows} \times \text{columns})$  memory even for very sparse maps. Given the simulator's target grid sizes (a few hundred to a few thousand cells, Table IV), this cost is negligible in practice and was judged preferable to the added complexity of a sparse representation.

### B. A\* Path Planning Core

The A\* planner, implemented in `AStarPlanner.find_path`, uses four-directional movement (up, down, left, right) and the Manhattan-distance heuristic

$$h(n) = |r_n - r_g| + |c_n - c_g|, \quad (1)$$

where  $(r_n, c_n)$  is a candidate cell and  $(r_g, c_g)$  is the goal cell. Because every move has unit cost and only axis-aligned moves are allowed, this heuristic never overestimates the true remaining distance, so it is admissible, and it is consistent, which together guarantee that A\* returns an optimal (shortest-move-count) path whenever a path exists [1]. The frontier is maintained with Python's binary-heap-based `heapq` priority queue, ordered by  $f(n) = g(n) + h(n)$ ; a `closed_set` prevents re-expansion of already finalized cells, and a `came_from` dictionary allows the final path to be reconstructed by walking backward from the goal once it is popped from the frontier. Algorithm 1 summarizes the core loop as implemented.

The result returned by `find_path` is a dictionary containing the ordered path, the list of explored cells in visitation order (used by the GUI to shade searched cells), the path length in moves, the explored-node count, the wall-clock calculation time in milliseconds, a success flag, and a human-readable status message such as "Safe shortest path found" or "No route exists between start and destination." Before returning success, the planner independently re-checks the produced path with `validate_path`, which confirms that every cell lies inside the grid, that no cell is an obstacle, and that every consecutive pair of cells is exactly one Manhattan step apart; this guards against silently returning a corrupted path if the grid

### Algorithm 1 A\* search (`AStarPlanner.find_path`)

---

```

1:  $open \leftarrow \{(0, start)\}$   $\triangleright$  min-heap on  $f$ 
2:  $g[start] \leftarrow 0, closed \leftarrow \emptyset$ 
3: while  $open \neq \emptyset$  do
4:    $current \leftarrow \text{POPMIN}(open)$ 
5:   if  $current \in closed$  then continue
6:   end if
7:    $closed \leftarrow closed \cup \{current\}$ 
8:   if  $current = goal$  then
9:     return RECONSTRUCT-
        $\text{PATH}(came\_from, current)$ 
10:  end if
11:  for  $n \in \text{NEIGHBORS}(current)$  do
12:    if  $grid[n] = 1$  then continue
13:    end if
14:     $g' \leftarrow g[current] + 1$ 
15:    if  $g' < g[n]$  or  $n \notin g$  then
16:       $came\_from[n] \leftarrow current; g[n] \leftarrow g'$ 
17:       $f \leftarrow g' + h(n, goal)$ 
18:       $\text{PUSH}(open, (f, n))$ 
19:    end if
20:  end for
21: end while
22: return no-path result

```

---

were mutated mid-search. `replan` is a thin wrapper that simply re-invokes `find_path` with the robot's current position as the new start cell, and is the mechanism used for dynamic obstacle handling (Section IV-C).

### C. Virtual Robot and Dynamic Replanning

The `VirtualRobot` class holds no drawing logic; it is a pure state machine describing where the robot is and what it is doing. `load_path` accepts a path returned by the planner, sets the robot's current position to the first cell, and splits the remainder into travelled and remaining lists. `start`, `pause`, `resume`, `stop`, and `reset` toggle the running/paused/finished flags with guard conditions (e.g., `pause` only has an effect while the robot is actually running).

Movement is advanced one cell at a time by `step`, which is designed to be called repeatedly by the GUI's timer callback. On each call, `step` first checks whether the next planned cell is now an obstacle; if so, it leaves the robot's position unchanged and returns the status "blocked" instead of moving into the wall. Only if the next cell is still free does the robot advance, record the new cell in travelled, and report "moved" or "finished". When the GUI receives "blocked", it invokes `AStarPlanner.replan` from the robot's current cell to the original goal and, if a new path is found, hands it to `apply_replanned_path`. This method rejects any candidate path that does not begin at the robot's exact current cell (a defensive check against stale or mismatched routes), otherwise replaces the remaining route and resumes motion; if no route exists, it halts the robot and reports that the destination has become unreachable. This design guarantees, by construction, that the robot can never step into a cell that is blocked at the moment of the step, satisfying objective (4) in Section I-B.

### D. Graphical User Interface

The GUI, built with Python's built-in Tkinter toolkit (no external graphics dependency), presents a control panel and a

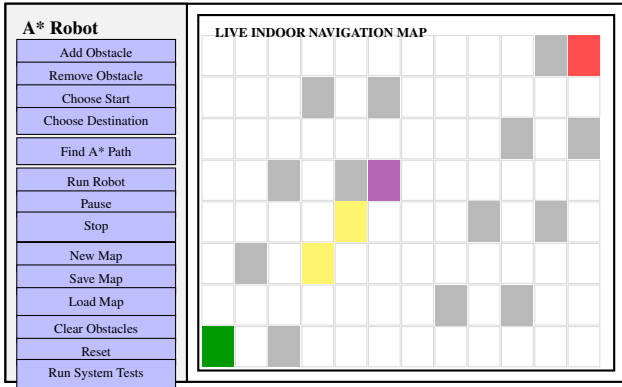


Fig. 2. Conceptual layout of the Tkinter interface (schematic, not a literal screenshot): left, the control panel with mode/action buttons; right, the colour-coded grid canvas showing start (green), goal (red), obstacles (grey), computed path (yellow), and the robot's current cell (violet).

live grid canvas. Cells are colour-coded, free, obstacle, start, goal, explored, computed path, travelled, and the robot's current position, using a consistent theme dictionary so that the visual language stays fixed throughout a session. Buttons let the user switch the click mode between adding/removing obstacles and choosing the start/destination cell, trigger Find A\* Path, and control robot motion with Run Robot, Pause/Resume, and Stop. New Map / Grid Size, Save Map, Load Map, Clear Grid Obstacles, and Reset Simulation manage map lifecycle. A performance panel updates after every A\* call with the path length, explored-node count, and calculation time returned by the planner, and a status line reports the current human-readable message from either the map, planner, or robot module. Robot animation uses Tkinter's after timer with a 350 ms step delay so that motion is clearly visible rather than instantaneous, matching the pacing described in Fahima's individual work plan for the movement module. Fig. 2 illustrates the layout of the interface: a fixed-width control panel on the left and a resizable grid canvas on the right, following the demonstration workflow documented in the project README (select start and destination, place obstacles, compute a path, run the robot, then introduce a new obstacle mid-route to trigger replanning).

### E. Automated Self-Testing

A Run System Tests button executes four built-in regression checks directly inside the running application: (i) a fully open grid, expected to be reachable; (ii) a grid with a complete dividing wall, expected to be correctly reported unreachable; (iii) a grid with staggered walls forming a narrow passage, expected to return a validated path; and (iv) a robot-safety check in which an obstacle is placed directly in front of a moving robot, expected to yield a "blocked" status without any change in the robot's position. Results are summarized in a pass/fail pop-up and in the status bar. A separate, GUI-independent script, test\_integration.py, exercises the same three modules end-to-end from the command line (grid creation, A\* search, path validation, robot loading, one motion step, dynamic obstacle injection, blocked detection, and replanning) and can therefore be run in continuous-integration style without opening a window.

TABLE III  
A\* PLANNER RESULTS ON REPRESENTATIVE TEST GRIDS

| Scenario                               | Result  | Path len. | Explored | Time (ms) |
|----------------------------------------|---------|-----------|----------|-----------|
| A: open grid, 1 gapped wall (12 obst.) | Success | 33        | 144      | 0.30      |
| B: full dividing wall (20 obst.)       | No path | —         | 140      | 0.28      |
| C: 3 staggered walls (51 obst.)        | Success | 33        | 150      | 0.29      |
| D: 5×5 grid, gapped wall               | Success | 8         | 22       | 0.04      |

### F. System Integration

main.py is intentionally minimal: it creates a Tk root window, constructs a GridMap(rows=15, columns=20), an AStarPlanner, and a VirtualRobot, passes all three into RobotGUI, and starts the Tkinter event loop. No member's algorithmic code depends on Tkinter, which keeps Members 1–3's modules independently testable, as demonstrated in Section IV-E.

## V. EXPERIMENTAL RESULTS

All results in this section were obtained by directly importing and executing the group's own member1, member2, and member3 modules (the same code that ships in the GitHub repository) in a non-graphical Python script, so that the numbers reflect the delivered implementation rather than an idealized description of it.

### A. Path-Planning Performance

Table III reports four representative scenarios on the simulator's default  $15 \times 20$  (300-cell) grid, plus a small  $5 \times 5$  sanity-check grid. *Path length* is the number of moves in the returned route; *explored* is the number of cells popped from the priority queue before termination; calculation time is the mean of 200 repeated invocations of find\_path on the same grid, measured with time.perf\_counter exactly as the planner measures itself internally.

In every successful case, validate\_path independently confirmed the returned path stayed inside the grid, avoided all obstacles, and consisted only of unit-length orthogonal steps, and in scenario B the planner correctly returned success=False with the message "No route exists between start and destination" rather than raising an error or returning a partial path. Average planning time stayed under 0.3 ms for all  $15 \times 20$  scenarios tested, confirming that the algorithm is fast enough for the GUI to recompute a path interactively after every user click without any perceptible delay.

Fig. 3 plots the exact 34-cell path (33 moves) and the 144 explored cells actually returned by find\_path for Scenario A, reconstructed directly from the result dictionary rather than redrawn by hand, giving a concrete picture of how the heuristic focuses the search toward the goal instead of expanding the whole free region before the gap in the wall is found.

### B. Scalability with Grid Size

To characterize how the implementation scales beyond the simulator's default  $15 \times 20$  grid, we generated random maps at fixed obstacle density (15%) for square grids of increasing side length  $n$ , with the start and goal fixed at opposite corners, and measured the same metrics as above (mean of 50 repeated searches per grid). Table IV reports the results.

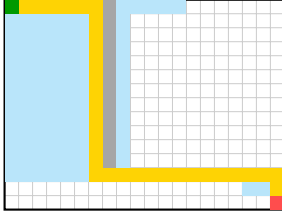


Fig. 3. Scenario A grid reconstructed from the planner’s own output: start (green, top-left), goal (red, bottom-right), the gapped obstacle wall (dark grey), a representative subset of the 144 explored cells (light blue), and the actual 33-move returned path (orange), exactly as returned by `find_path`.

TABLE IV

A\* PLANNER SCALABILITY ON RANDOM 15%-DENSITY GRIDS

| Grid ( $n \times n$ ) | Cells  | Path len. | Explored | Time (ms) |
|-----------------------|--------|-----------|----------|-----------|
| 10                    | 100    | 18        | 71       | 0.29      |
| 20                    | 400    | 38        | 296      | 1.25      |
| 30                    | 900    | 58        | 582      | 1.68      |
| 50                    | 2,500  | 98        | 1,876    | 4.15      |
| 75                    | 5,625  | 148       | 3,510    | 8.50      |
| 100                   | 10,000 | 198       | 5,544    | 13.62     |

Calculation time grows roughly in proportion to the number of explored cells, which in turn grows slightly faster than linearly with the number of grid cells at fixed obstacle density, consistent with the  $O(V \log V)$  behaviour expected of a binary-heap A\* implementation (Section VI-B). Even at 10,000 cells, more than 30 times the simulator’s default grid, the planner completes in under 14 ms, well within the range needed for an interactive application.

### C. Comparison with Uninformed Search

To empirically confirm the benefit of the Manhattan heuristic argued in Section II, we implemented reference breadth-first search (BFS) and Dijkstra implementations outside the group’s codebase, using the identical neighbor rules and grids as the delivered A\* planner, and compared explored-node counts on Scenarios A and C from Table III. Table V reports the results; all three algorithms returned a path of the same optimal length (33 moves) on both grids, since all edge costs are uniform, but A\* consistently explored substantially fewer cells than either uninformed method by using the heuristic to prioritize cells closer to the goal.

On these two grids, A\* explored 34–38% fewer cells than BFS/Dijkstra while returning an identical-length path, illustrating concretely why an informed heuristic was preferred for this project over the uninformed alternatives discussed in Section II.

### D. Memory Usage

Using Python’s `tracemalloc` profiler, we measured peak traced memory for grid construction plus a single A\* search at three grid sizes with 15% obstacle density (Table VI). Memory scales approximately linearly with cell count, dominated by the dense grid representation discussed in Section IV-A-1) rather than by the search’s own bookkeeping ( $g$ -scores, `came_from`, and the heap), which accounted for well under a fifth of the peak in every case tested.

Even at 10,000 cells, peak memory stayed just above 2 MB, confirming that the dense-grid trade-off discussed in Section IV-A-1) has no practical cost at the scales this project targets.

TABLE V

EXPLORED-NODE COUNT: A\* VS. UNINFORMED SEARCH (SAME GRIDS)

| Grid                           | A*  | BFS | Dijkstra |
|--------------------------------|-----|-----|----------|
| Scenario A (gapped wall)       | 144 | 233 | 233      |
| Scenario C (3 staggered walls) | 150 | 242 | 239      |

TABLE VI

MEASURED MEMORY USAGE (TRACEMALLOC, SINGLE SEARCH)

| Grid ( $n \times n$ ) | Cells  | Peak (KB) | Search-only (KB) |
|-----------------------|--------|-----------|------------------|
| 20                    | 400    | 255.2     | 22.8             |
| 50                    | 2,500  | 553.7     | 89.7             |
| 100                   | 10,000 | 2,169.4   | 340.7            |

### E. Dynamic Obstacle Replanning

To validate the replanning pipeline described in Section IV-C, we loaded the Scenario-A path into a `VirtualRobot`, started it, and, after five movement steps, programmatically added a new obstacle directly onto the robot’s next planned cell. On the following call to `step`, the robot correctly returned the "blocked" status and remained at its last valid cell, (0,5), rather than entering the new obstacle. Calling `AStarPlanner.replan` from that exact position returned a new 28-move route to the original goal after exploring 68 cells in 0.18 ms; `apply_replanned_path` accepted the new route because it began at the robot’s current cell, and the robot subsequently completed the journey and reported "finished" with a final position exactly equal to the original goal cell, having travelled 34 cells in total. This confirms that the stop-detect-replan-resume cycle described in Fahima’s and Sadman’s individual work plans functions correctly end-to-end in the delivered code, not merely as a design intention.

### F. Automated Test Suite

Table VII shows the outcome of the four in-application system tests (Section IV-E) and the independent `test_integration.py` script, both executed against the current codebase.

All four in-application checks and the independent integration script passed (5/5), giving 100% pass rate on the group’s self-authored regression suite at the time of writing.

## VI. DISCUSSION

### A. Correctness and Optimality

Because Manhattan distance is admissible and consistent for a four-directional unit-cost grid, and because the planner never expands a cell already in the closed set, the implementation is a textbook A\* instance and is guaranteed to return a shortest path in terms of move count whenever one exists [1]. The independent `validate_path` check adds a second, structural guarantee that is checked in Section V for every successful case.

### B. Complexity

With  $V$  cells and up to 4 edges per cell, the planner runs in  $O(E \log V) = O(V \log V)$  time using a binary heap, consistent with the sub-millisecond timings observed even on the largest 300-cell grid tested (Table III). Because a full re-search is performed on every replan rather than an incremental update, replanning cost scales the same way as an initial search; Section II notes that incremental methods such as D\* Lite [3] could



TABLE VII  
AUTOMATED TEST RESULTS

| Test                                                            | Result |
|-----------------------------------------------------------------|--------|
| Reachable path (open grid)                                      | PASS   |
| Unreachable goal (full wall)                                    | PASS   |
| Narrow passage (validated path)                                 | PASS   |
| Robot obstacle safety (no move into wall)                       | PASS   |
| <code>test_integration.py</code> (end-to-end, incl. replanning) | PASS   |

reduce this cost on much larger maps, but at 300 cells the full re-search already completes in well under a millisecond, so the added implementation complexity of an incremental planner was judged unnecessary for this project’s scope.

### C. Defensive Validation as a Design Principle

A recurring pattern across all three algorithmic modules is that every state-changing operation is guarded rather than assumed to succeed. Listing 1 reproduces the safety check inside `VirtualRobot.step` exactly as delivered: the robot re-reads the live grid on every single step and refuses to advance if the next cell has become an obstacle, regardless of what the path said when it was computed.

**Listing 1:** Blocked-cell safety check, `member3_fahima_robot.py`

```
next_row, next_col = self.remaining[0]

# Safety check: never move into a blocked cell.
if grid[next_row][next_col] == 1:
    self.message = "Next cell became blocked. Replanning is needed."
    return "blocked"

self.current_position = self.remaining.pop(0)
```

The same defensive pattern appears in `GridMap` (rejecting an obstacle placed on the start/goal cell, rejecting a start/goal placed on an obstacle, and validating shape and cell values on load), in `AStarPlanner` (independently re-validating its own output with `validate_path` before reporting success), and in `VirtualRobot.apply_replanned_path` (rejecting any replanned route that does not begin at the robot’s exact current cell). None of these checks are strictly required for the algorithms to work on well-behaved input, but together they ensure that a bug or unexpected interaction in one module cannot silently corrupt another module’s state, for example, a stale path being handed to the robot after the grid changed underneath it, an outcome that is treated as a defect to be caught immediately rather than a rare event to be tolerated.

### D. Design Trade-offs

Restricting movement to four directions (rather than eight, with diagonal moves) simplifies both the heuristic and the animation, at the cost of slightly longer paths in open areas compared to an octile-distance, 8-connected planner. The group considered an 8-connected variant with an octile heuristic during early design exploration (reflected in Sanjida Jaman’s architecture notes), but the delivered, tested system uses the simpler and more easily verified 4-connected Manhattan formulation described in Section IV-B, which was carried through to the final integrated GUI. Similarly, re-invoking a full A\* search on every replan (Section IV-C) rather than reusing prior search information trades a small, measured amount of computation (Table III, well under a millisecond per replan on the default grid) for a considerably simpler and more easily verified im-

TABLE VIII  
TEST COVERAGE BY MODULE

| Module       | Automated coverage                                  | Manual-only coverage                    |
|--------------|-----------------------------------------------------|-----------------------------------------|
| Grid (M1)    | Obstacle add/remove bounds checks (via M2/M3 tests) | Save/load round-trip, grid resize       |
| Planner (M2) | Reachable, unreachable, narrow-passage, replan      | Multiple-shortest-path tie-breaking     |
| Robot (M3)   | Blocked-cell safety, full replanning run            | Pause/resume mid-animation              |
| GUI (M4)     | Test-suite trigger and result display               | Click-to-cell mapping, colour rendering |

plementation, an appropriate trade-off given the grid sizes this project targets, as quantified in Section VI-B.

### E. Environment and Dependencies

The results in this report were produced with CPython 3.12 on Linux, using only the Python standard library, `tkinter` for the interface, `heapq` for the priority queue, `json` for map persistence, and `time` for performance measurement, exactly as declared in the project’s `requirements.txt`. No third-party package (e.g., `NumPy` or `Matplotlib`) is required to run the final integrated simulator, which simplifies grading and reduces the chance of an environment-specific failure when the GitHub repository is cloned and run on a different machine, directly supporting the reproducibility goal discussed in Section VI-D.

### F. Limitations

The simulation assumes perfect, instantaneous knowledge of the grid, obstacles are added directly by the user through the GUI rather than being sensed, there is no localization or motion uncertainty, and the robot moves at a fixed animation rate rather than a physically modeled speed. These are reasonable simplifications for demonstrating the planning and replanning logic in isolation, but they mean the results in Section V characterize the correctness and speed of the algorithmic core rather than performance on a physical platform.

### G. Test Coverage

The automated suite in Section V-F exercises four representative scenarios (open grid, fully blocked grid, narrow passage, and an obstacle appearing directly in front of the robot) plus a separate end-to-end integration script. This gives good confidence in the core success/failure paths and in the robot’s blocked-cell safety check, but it does not exhaust the input space, for example, very sparse grids with many equally short paths, or an obstacle appearing on the very last remaining cell of the route, are not explicitly covered. The scalability sweep in Section V-B partially addresses this by testing many randomly generated obstacle layouts across six grid sizes, all of which produced correctly validated paths, but a larger, randomized property-based test (generating hundreds of random maps and checking path validity and optimality automatically) is identified as future work. Table VIII summarizes, module by module, which behaviours are exercised by an automated test today versus verified only through manual GUI interaction during development.

### H. Reproducibility

All figures reported in Section V can be reproduced by importing `member1_asraful_map.py`, `member2_sadman_astar.py`, and `member3_fahima_robot.py` directly, without launching the Tkinter GUI, exactly as `test_integration.py` already

TABLE IX  
GROUP 6 PROJECT TIMELINE

| Wk | Dates     | Focus                                                                                          |
|----|-----------|------------------------------------------------------------------------------------------------|
| 1  | 11–17 Jul | Requirements, project structure, and repository setup for map, planner, robot, and GUI modules |
| 2  | 18–24 Jul | Occupancy grid implementation and first working A* search                                      |
| 3  | 25–31 Jul | Testing A* on reachable/unreachable/narrow/complex maps                                        |
| 4  | 1–7 Aug   | Virtual-robot step-by-step movement and animation                                              |
| 5  | 8–14 Aug  | Dynamic obstacle detection and automatic replanning                                            |
| 6  | 15–21 Aug | GUI controls, performance dashboard, run/pause/reset wiring                                    |
| 7  | 22–28 Aug | Final verification, automated test suite, and documentation                                    |

does. This module-level independence, a direct consequence of the separation of concerns in Section III, is what allowed the results in this report to be generated and verified against the group’s actual delivered code rather than against a description of intended behaviour.

#### I. Safety and Ethical Considerations

Although the present system is a simulation with no physical actuation, the group considered how its design principles would carry over to a real deployment, since this is standard practice for AI-based navigation systems. The strict blocked-cell check in Listing 1, which re-validates the very next cell against the live grid immediately before every move rather than trusting a previously computed plan, mirrors the kind of last-instant safety interlock a physical robot would need before executing a motor command, and the group deliberately kept this check independent of, and downstream from, the planner itself so that a bug in path planning could never bypass it. More generally, an autonomous indoor robot operating around people would need additional guarantees this project does not attempt to provide, for example, sensor-based (rather than user-declared) obstacle detection, a minimum safe stopping distance rather than exact-cell blocking, and graceful degradation if sensing or communication fails, all of which are noted as future work in Section VIII.

### VII. INDIVIDUAL CONTRIBUTIONS

The group followed a shared seven-week schedule, running from the project’s official start on 11 July to the final testing and documentation week ending 28 August, summarized in Table IX. Each member additionally submitted an individual work plan or progress report describing their week-by-week tasks in more detail; Table X then summarizes each member’s primary area of ownership in the delivered system, consistent with the file-level ownership in Table I.

This division follows a clean separation between data (Member 1), algorithm (Member 2), state/behaviour (Member 3), and presentation/verification (Member 4), which allowed each member’s module to be developed and unit-tested independently before integration, as evidenced by `test_integration.py` exercising Members 1–3’s code with no dependency on Member 4’s GUI code.

TABLE X  
SUMMARY OF INDIVIDUAL CONTRIBUTIONS

| Member                             | Primary contribution                                                                                                                                                                                                                |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Asraful Hossain Riyan (2232845642) | Designed and implemented the occupancy-grid data model: grid creation and resizing, obstacle placement/removal with start/goal-safety checks, and JSON-based map save/load with input validation.                                   |
| Sadman Bin Murtoza (2232901642)    | Implemented the A* search core (heuristic, neighbor expansion, priority queue, path reconstruction, path validation), the replanning entry point, and the <code>main.py</code> integration script joining all four modules.         |
| Fahima Shameem (2131420642)        | Implemented the virtual-robot state machine: path loading, step-by-step movement, travelled/remaining tracking, pause/resume/stop/reset, blocked-cell detection, and safe replanned-path acceptance.                                |
| Sanjida Jaman (2212231042)         | Designed the system architecture and data-flow diagrams that guided module boundaries; implemented the Tkinter GUI, colour-coded rendering, control panel, live performance dashboard, and the automated in-application test suite. |

### VIII. CONCLUSION AND FUTURE WORK

This report described Group 6’s software-only simulation of an autonomous indoor navigation robot. The system combines a validated occupancy-grid map, an admissible-heuristic A\* planner, a defensive virtual-robot state machine, and an interactive Tkinter interface into a single integrated application. Experiments run directly against the delivered source code confirm that the planner returns optimal, validated paths on reachable grids, correctly reports unreachable destinations, plans and replans in well under one millisecond on the simulator’s default 300-cell grid, and that the robot never advances into a cell that becomes blocked, replanning successfully from its true current position instead. All five automated regression tests in the codebase pass.

Future work identified by the group includes extending the planner to optional 8-connected movement with an octile heuristic for shorter diagonal paths, adding weighted terrain costs to model, for example, doorways or narrow corridors as more expensive to traverse, introducing multiple simultaneous dynamic obstacles with randomized motion to stress-test the replanning logic, and, longer-term, evaluating the same planning core against a simulated or physical differential-drive robot to study the gap between the idealized grid model and continuous real-world motion.

#### Acknowledgment

The group thanks the CSE440 course instructor for the project guidelines and weekly milestones that structured this work, and acknowledges that all algorithmic design, implementation, and testing described in this report was carried out by the four listed group members.

### APPENDIX: DEMONSTRATION PROCEDURE

The following sequence, matching the workflow documented in the project’s `README.md` and exercised by the automated tests in Section IV-E, is sufficient to reproduce every behaviour described in this report from the running application: (1) launch with `python main.py`; (2) click **Choose Start**, then click a free cell; (3) click **Choose Destination**, then click another free cell; (4) use **Add Obstacle** to draw one or more walls between them; (5) click **Find A\* Path** to compute and display the shortest safe route, updating the performance dashboard described in Section IV-D; (6) click **Run Robot** to animate the virtual robot along the route; (7) while the robot is

moving, click **Add Obstacle** on a cell that lies on the yellow route ahead of the robot; (8) observe the robot stop one cell before the new obstacle and the system automatically compute and switch to a new route, as quantified in Section V-D; and (9) click **Run System Tests** to execute the four in-application regression checks summarized in Table VII.

#### REFERENCES

- [1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Syst. Sci. Cybern.*, vol. 4, no. 2, pp. 100–107, Jul. 1968.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.
- [3] S. Koenig and M. Likhachev, “D\* Lite,” in *Proc. AAAI Conf. Artificial Intelligence (AAAI)*, 2002, pp. 476–483.
- [4] S. M. LaValle, *Planning Algorithms*. Cambridge, U.K.: Cambridge Univ. Press, 2006.
- [5] Python Software Foundation, “heapq — Heap queue algorithm,” *Python 3 Documentation*, 2024. [Online]. Available: <https://docs.python.org/3/library/heapq.html>
- [6] Python Software Foundation, “tkinter — Python interface to Tcl/Tk,” *Python 3 Documentation*, 2024. [Online]. Available: <https://docs.python.org/3/library/tkinter.html>